2A

最大用时应该为0.5s，
令用时为0.5s，即$10n^2 \times 10^{-8} = 0.5$,
解得$n \approx 2236$,
即最多为n准备2236条数据

2B

令用时为0.5s，即$20n \log_2 n \times 10^{-8} = 0.5$,
解得$n \approx 145746$
即最多为n准备145746条数据

3A

(1)sum每次计算后没有重置，后续结果错误
(2)当数据较大时，$2000 \times 2000 \times 10^5$超过int的存储范围，会溢出
(3)solution_2.cpp中，sum += rowsum[x + j][y + b] - rowsum[x + j][y]错误，也改为sum += rowsum[x + j -1][y + b] - rowsum[x + j -1][y]

均通过静态检查发现

AIGC能找到(1)(2)的问题，当单纯让AIGC寻找一个较大代码的问题时，对于逻辑上较细节的地方aigc容易忽略，需要优化提示词，此外感觉aigc的代码能力“波动”比较大

3B

编译选项需要添加-g
命令行调试时，需要用gdb ./(生成文件名)加载可执行文件，之后用start，next，step等命令运行
图形化调试工具即点击运行和调试按钮，快捷键F5，断点的添加可以直接在代码左边点击，F5快捷键可以运行到下一个断点，变量会在变量栏内显示

3C

获取时间作为随机数生成的种子，使每次生成的随机数不同

3D

system("g++ xxxxx.cpp -o xxxxx");
编译一个cpp文件，生成一个可执行文件

system("./rand_input > rand.in");
运行rand_input文件，并将输出输入到rand.in文件中

system("./check_input < rand.in");
运行 check_input 程序，以 rand.in 作为输入，

system("./solution_1 < rand.in > 1.out");
运行solution_1，以 rand.in 作为输入，将输出结果输入到 1.out 文件

system("diff 1.out 2.out");
对比两个文件内容

3E

4×10^{11}，需要long long

4A

算法思路：提前存储从(1,1)到(x,y)的矩阵元素的和，从而在访问时直接利用存储的数据加减得到

4B

实验序号	solution_1	solution_2	solution_3	n m q
1	2.2	0.14	0.1	500 500 80000
2	25	0.8	0.55	500 500 800000
3	0.31	0.041	0.045	500 500 8000
4	2.6	0.4	0.45	1500 1500 8000
5	278.685	2.089	0.971	1500 1500 80000
6	0.005	0.004	0.005	100 100 800

上表为平均用时

4C

当查询次数较少或者矩阵较小时，区别不大；
当查询次数很大且矩阵较大时，solution_3会有明显优势

5A

生成的代码：

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>二维前缀和算法可视化 | Solution 3</title>
    <script src="https://cdn.tailwindcss.com"></script>
    <link href="https://cdn.jsdelivr.net/npm/font-awesome@4.7.0/css/font-awesome.min.css" rel="s

    <!-- 配置Tailwind自定义颜色和字体 -->
    <script>
        tailwind.config = {
            theme: {
                extend: {
                    colors: {
                        primary: '#3B82F6',
                        secondary: '#10B981',
                        accent: '#8B5CF6',
                        dark: '#1E293B',
                        light: '#F8FAFC'
                    },
                    fontFamily: {
                        sans: ['Inter', 'system-ui', 'sans-serif'],
                    },
                }
            }
        }
    </script>

    <style type="text/tailwindcss">
        @layer utilities {
            .matrix-cell {
                @apply w-12 h-12 flex items-center justify-center border border-gray-300 transit
            }
            .matrix-cell-highlight {
                @apply bg-primary/20 border-primary;
            }
            .matrix-cell-selected {
                @apply bg-secondary/30 border-secondary;
            }
            .step-active {
                @apply bg-primary text-white border-primary;
            }
```

```
            .shadow-soft {
                @apply shadow-md hover:shadow-lg transition-shadow duration-300;
            }
        }
    </style>
</head>
<body class="bg-gray-50 min-h-screen font-sans text-dark">
    <!-- 顶部导航 -->
    <header class="bg-white shadow-md sticky top-0 z-50">
        <div class="container mx-auto px-4 py-4 flex justify-between items-center">
            <div class="flex items-center space-x-2">
                <i class="fa fa-calculator text-primary text-2xl"></i>
                <h1 class="text-xl md:text-2xl font-bold text-dark">二维前缀和算法可视化</h1>
            </div>
            <div class="hidden md:flex items-center space-x-6">
                <a href="#about" class="text-gray-600 hover:text-primary transition-colors">关于
                <a href="#visualization" class="text-gray-600 hover:text-primary transition-colo
                <a href="#code" class="text-gray-600 hover:text-primary transition-colors">代码解
            </div>
            <button class="md:hidden text-gray-600" id="menuToggle">
                <i class="fa fa-bars text-xl"></i>
            </button>
        </div>
        <!-- 移动端菜单 -->
        <div id="mobileMenu" class="hidden md:hidden bg-white border-t border-gray-200 px-4 py-2
            <a href="#about" class="block py-2 text-gray-600 hover:text-primary">关于算法</a>
            <a href="#visualization" class="block py-2 text-gray-600 hover:text-primary">可视化<
            <a href="#code" class="block py-2 text-gray-600 hover:text-primary">代码解析</a>
        </div>
    </header>

    <main class="container mx-auto px-4 py-8">
        <!-- 关于算法部分 -->
        <section id="about" class="mb-16">
            <div class="bg-white rounded-xl shadow-soft p-6 md:p-8">
                <h2 class="text-2xl font-bold mb-4 text-dark flex items-center">
                    <i class="fa fa-info-circle text-primary mr-2"></i>关于二维前缀和算法
                </h2>
                <p class="text-gray-700 mb-4">
                    二维前缀和算法是一种高效计算二维数组中任意子矩阵元素和的技术。通过预先计算前缀和矩
                </p>
                <div class="grid md:grid-cols-2 gap-6 mt-6">
                    <div class="bg-blue-50 p-5 rounded-lg border border-blue-100">
```

```
                <h3 class="font-semibold text-lg mb-2 text-primary">算法优势</h3>
                <ul class="list-disc pl-5 text-gray-700 space-y-1">
                    <li>预处理时间复杂度：O(n*m)</li>
                    <li>每次查询时间复杂度：O(1)</li>
                    <li>适合多次查询的场景</li>
                    <li>内存占用小，只需额外存储两个辅助矩阵</li>
                </ul>
            </div>
            <div class="bg-green-50 p-5 rounded-lg border border-green-100">
                <h3 class="font-semibold text-lg mb-2 text-secondary">主要步骤</h3>
                <ol class="list-decimal pl-5 text-gray-700 space-y-1">
                    <li>读取输入矩阵</li>
                    <li>计算每行的前缀和(rowsum)</li>
                    <li>计算二维前缀和矩阵(matrixsum)</li>
                    <li>通过公式快速计算任意子矩阵的和</li>
                </ol>
            </div>
        </div>
    </div>
</section>

<!-- 可视化部分 -->
<section id="visualization" class="mb-16">
    <div class="bg-white rounded-xl shadow-soft p-6 md:p-8">
        <h2 class="text-2xl font-bold mb-6 text-dark flex items-center">
            <i class="fa fa-eye text-primary mr-2"></i>算法可视化演示
        </h2>

        <!-- 控制面板 -->
        <div class="mb-8 bg-gray-50 p-4 rounded-lg border border-gray-200">
            <div class="grid md:grid-cols-3 gap-4 mb-4">
                <div>
                    <label class="block text-sm font-medium text-gray-700 mb-1">矩阵大小
                    <div class="flex space-x-2">
                        <input type="number" id="nInput" value="3" min="2" max="10" cla
                        <span class="flex items-center">×</span>
                        <input type="number" id="mInput" value="3" min="2" max="10" cla
                    </div>
                </div>
                <div>
                    <label class="block text-sm font-medium text-gray-700 mb-1">查询数量
                    <input type="number" id="qInput" value="2" min="1" max="5" class="w
                </div>
```

```
            <div>
                <label class="block text-sm font-medium text-gray-700 mb-1">操作</la
                <button id="generateBtn" class="w-full bg-primary hover:bg-primary/9
                    <i class="fa fa-refresh mr-1"></i> 生成新矩阵
                </button>
            </div>
        </div>

        <!-- 步骤导航 -->
        <div class="flex flex-wrap gap-2 mt-4">
            <button id="step1Btn" class="step-btn step-active px-4 py-2 rounded-md b
                1. 原始矩阵
            </button>
            <button id="step2Btn" class="step-btn px-4 py-2 rounded-md border borde
                2. 行前缀和 (rowsum)
            </button>
            <button id="step3Btn" class="step-btn px-4 py-2 rounded-md border borde
                3. 二维前缀和 (matrixsum)
            </button>
            <button id="step4Btn" class="step-btn px-4 py-2 rounded-md border borde
                4. 查询计算
            </button>
        </div>
    </div>

    <!-- 查询控制 (步骤4时显示) -->
    <div id="queryControls" class="mb-6 hidden">
        <div class="bg-purple-50 p-4 rounded-lg border border-purple-100">
            <h3 class="font-semibold text-lg mb-3 text-accent">查询参数设置</h3>
            <div class="grid md:grid-cols-4 gap-4">
                <div>
                    <label class="block text-sm font-medium text-gray-700 mb-1">起始
                    <input type="number" id="xInput" value="1" min="1" class="w-full
                </div>
                <div>
                    <label class="block text-sm font-medium text-gray-700 mb-1">起始
                    <input type="number" id="yInput" value="1" min="1" class="w-full
                </div>
                <div>
                    <label class="block text-sm font-medium text-gray-700 mb-1">行数
                    <input type="number" id="aInput" value="2" min="1" class="w-full
                </div>
                <div>
```

```
                        <label class="block text-sm font-medium text-gray-700 mb-1">列数
                        <input type="number" id="bInput" value="2" min="1" class="w-ful
                    </div>
                </div>
                <button id="calculateQueryBtn" class="mt-4 bg-accent hover:bg-accent/90
                    <i class="fa fa-calculator mr-1"></i> 计算区域和
                </button>
            </div>
        </div>

        <!-- 矩阵显示区域 -->
        <div class="overflow-x-auto">
            <div id="matrixContainer" class="flex justify-center my-8"></div>
        </div>

        <!-- 结果和解释 -->
        <div id="explanation" class="bg-gray-50 p-5 rounded-lg border border-gray-200">
            <h3 class="font-semibold text-lg mb-2 text-primary">当前步骤解释</h3>
            <p id="explanationText" class="text-gray-700">
                这是初始矩阵，我们将基于此计算行前缀和、二维前缀和，并进行区域和查询。点击"生
            </p>
            <div id="formulaDisplay" class="mt-4 hidden">
                <div class="bg-white p-3 rounded border border-gray-200">
                    <h4 class="font-medium text-gray-800 mb-2">区域和计算公式：</h4>
                    <div class="text-center py-2 text-lg">
                        sum = matrixsum[x+a-1][y+b-1] - matrixsum[x+a-1][y-1] - matrixsu
                    </div>
                </div>
            </div>
            <div id="resultDisplay" class="mt-4 hidden">
                <div class="bg-green-50 p-3 rounded border border-green-200">
                    <h4 class="font-medium text-green-800 mb-2">计算结果：</h4>
                    <p class="text-xl font-semibold text-green-700" id="resultValue">区域
                </div>
            </div>
        </div>
    </div>
</section>

<!-- 代码解析部分 -->
<section id="code" class="mb-16">
    <div class="bg-white rounded-xl shadow-soft p-6 md:p-8">
        <h2 class="text-2xl font-bold mb-6 text-dark flex items-center">
```

代码解析
</h2>

<div class="bg-gray-900 text-gray-100 p-4 rounded-lg font-mono text-sm overflow-

```
<pre>#include &lt;cstdio&gt;

// 定义全局数组
int matrix[2000][2000];       // 原始矩阵
int rowsum[2000][2000];       // 行前缀和矩阵
long long matrixsum[2000][2000];  // 二维前缀和矩阵

int main() {
    int n, m, q;
    scanf("%d%d", &n, &m);

    // 1. 读取原始矩阵数据
    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= m; ++j) {
            scanf("%d", &matrix[i][j]);
        }
    }

    // 2. 计算行前缀和
    for (int i = 1; i <= n; ++i) {
        rowsum[i][0] = 0;  // 每行第0列初始化为0
        for (int j = 1; j <= m; ++j) {
            // 当前位置的行前缀和 = 前一列的行前缀和 + 当前元素值
            rowsum[i][j] = rowsum[i][j - 1] + matrix[i][j];
        }
    }

    // 3. 计算二维前缀和
    // 初始化第0行
    for (int i = 0; i <= m; ++i) {
        matrixsum[0][i] = 0;
    }

    for (int i = 1; i <= n; ++i) {
        matrixsum[i][0] = 0;  // 每列第0行初始化为0
        for (int j = 1; j <= m; ++j) {
            // 当前位置的二维前缀和 = 上一行的前缀和 + 当前行的前缀和
            matrixsum[i][j] = matrixsum[i-1][j] + rowsum[i][j];
        }
```

```
    }

    // 4. 处理查询
    scanf("%d", &q);
    long long sum = 0;
    for (int i = 1; i <= q; ++i) {
        sum = 0;
        int x, y, a, b;
        scanf("%d %d %d %d", &x, &y, &a, &b);

        // 核心公式：计算指定区域的和
        sum = matrixsum[x+a-1][y + b - 1]
            - matrixsum[x + a -1][y - 1]
            - matrixsum[x - 1][y + b - 1]
            + matrixsum[x-1][y-1];

        printf("%d\n", sum);
    }
    return 0;
}</pre>
                </div>

                <div class="mt-6 grid md:grid-cols-2 gap-6">
                    <div>
                        <h3 class="text-xl font-semibold mb-3 text-primary">核心思想</h3>
                        <p class="text-gray-700 mb-2">二维前缀和算法的核心是通过空间换时间，预先计
                        <p class="text-gray-700">这种方法特别适用于需要多次查询的场景，能够显著提高
                    </div>
                    <div>
                        <h3 class="text-xl font-semibold mb-3 text-primary">复杂度分析</h3>
                        <ul class="text-gray-700 space-y-2">
                            <li><span class="font-medium">时间复杂度：</span>预处理阶段为O(n×m)，
                            <li><span class="font-medium">空间复杂度：</span>O(n×m)，用于存储两个
                            <li><span class="font-medium">优势：</span>当查询次数q很大时，效率提升
                        </ul>
                    </div>
                </div>
            </div>
        </section>
    </main>

    <footer class="bg-dark text-white py-8">
        <div class="container mx-auto px-4">
```

```
        <div class="flex flex-col md:flex-row justify-between items-center">
            <div class="mb-4 md:mb-0">
                <div class="flex items-center">
                    <i class="fa fa-calculator text-primary text-xl mr-2"></i>
                    <span class="font-bold text-lg">二维前缀和算法可视化</span>
                </div>
                <p class="text-gray-400 text-sm mt-1">基于solution_3.cpp实现</p>
            </div>
            <div class="flex space-x-4">
                <a href="#" class="text-gray-400 hover:text-primary transition-colors">
                    <i class="fa fa-github text-xl"></i>
                </a>
                <a href="#" class="text-gray-400 hover:text-primary transition-colors">
                    <i class="fa fa-linkedin text-xl"></i>
                </a>
                <a href="#" class="text-gray-400 hover:text-primary transition-colors">
                    <i class="fa fa-twitter text-xl"></i>
                </a>
            </div>
        </div>
        <div class="border-t border-gray-800 mt-6 pt-6 text-center text-gray-500 text-sm">
            &copy; 2023 算法可视化演示 | 基于C++代码solution_3.cpp实现
        </div>
    </div>
</footer>

<script>
    // 全局变量
    let n = 3, m = 3, q = 2;
    let matrix = [];
    let rowsum = [];
    let matrixsum = [];
    let currentStep = 1;

    // DOM元素
    const nInput = document.getElementById('nInput');
    const mInput = document.getElementById('mInput');
    const qInput = document.getElementById('qInput');
    const generateBtn = document.getElementById('generateBtn');
    const matrixContainer = document.getElementById('matrixContainer');
    const explanationText = document.getElementById('explanationText');
    const stepBtns = document.querySelectorAll('.step-btn');
    const step1Btn = document.getElementById('step1Btn');
```

```
const step2Btn = document.getElementById('step2Btn');
const step3Btn = document.getElementById('step3Btn');
const step4Btn = document.getElementById('step4Btn');
const queryControls = document.getElementById('queryControls');
const xInput = document.getElementById('xInput');
const yInput = document.getElementById('yInput');
const aInput = document.getElementById('aInput');
const bInput = document.getElementById('bInput');
const calculateQueryBtn = document.getElementById('calculateQueryBtn');
const formulaDisplay = document.getElementById('formulaDisplay');
const resultDisplay = document.getElementById('resultDisplay');
const resultValue = document.getElementById('resultValue');
const menuToggle = document.getElementById('menuToggle');
const mobileMenu = document.getElementById('mobileMenu');

// 初始化
document.addEventListener('DOMContentLoaded', () => {
    generateMatrix();
    setupEventListeners();
});

// 设置事件监听器
function setupEventListeners() {
    // 生成新矩阵
    generateBtn.addEventListener('click', () => {
        n = parseInt(nInput.value) || 3;
        m = parseInt(mInput.value) || 3;
        q = parseInt(qInput.value) || 2;

        // 限制矩阵大小
        n = Math.min(Math.max(n, 2), 10);
        m = Math.min(Math.max(m, 2), 10);
        q = Math.min(Math.max(q, 1), 5);

        // 更新输入框
        nInput.value = n;
        mInput.value = m;
        qInput.value = q;

        generateMatrix();
        showStep(1);
    });
```

```
    // 步骤按钮
    step1Btn.addEventListener('click', () => showStep(1));
    step2Btn.addEventListener('click', () => showStep(2));
    step3Btn.addEventListener('click', () => showStep(3));
    step4Btn.addEventListener('click', () => showStep(4));

    // 计算查询
    calculateQueryBtn.addEventListener('click', calculateQuery);

    // 移动端菜单
    menuToggle.addEventListener('click', () => {
        mobileMenu.classList.toggle('hidden');
    });

    // 查询参数变化时更新最大值限制
    xInput.addEventListener('input', updateQueryInputLimits);
    yInput.addEventListener('input', updateQueryInputLimits);
    aInput.addEventListener('input', updateQueryInputLimits);
    bInput.addEventListener('input', updateQueryInputLimits);
}

// 生成随机矩阵
function generateMatrix() {
    // 初始化原始矩阵
    matrix = [];
    for (let i = 0; i <= n; i++) {
        matrix[i] = [];
        for (let j = 0; j <= m; j++) {
            if (i === 0 || j === 0) {
                matrix[i][j] = 0;  // 第0行和第0列都为0
            } else {
                matrix[i][j] = Math.floor(Math.random() * 10) + 1;  // 1-10的随机数
            }
        }
    }

    // 计算行前缀和
    rowsum = [];
    for (let i = 0; i <= n; i++) {
        rowsum[i] = [];
        rowsum[i][0] = 0;
        for (let j = 1; j <= m; j++) {
            if (i === 0) {
```

```
                rowsum[i][j] = 0;
            } else {
                rowsum[i][j] = rowsum[i][j - 1] + matrix[i][j];
            }
        }
    }

    // 计算二维前缀和
    matrixsum = [];
    for (let i = 0; i <= n; i++) {
        matrixsum[i] = [];
        for (let j = 0; j <= m; j++) {
            if (i === 0 || j === 0) {
                matrixsum[i][j] = 0;
            } else {
                matrixsum[i][j] = matrixsum[i - 1][j] + rowsum[i][j];
            }
        }
    }

    // 更新查询输入框的限制
    updateQueryInputLimits();

    // 显示当前步骤的矩阵
    renderMatrix();
}

// 更新查询输入框的限制
function updateQueryInputLimits() {
    const x = parseInt(xInput.value) || 1;
    const y = parseInt(yInput.value) || 1;

    // 设置最大值
    xInput.max = n;
    yInput.max = m;
    aInput.max = n - x + 1;
    bInput.max = m - y + 1;

    // 确保值在有效范围内
    if (x > n) xInput.value = n;
    if (y > m) yInput.value = m;
    if (parseInt(aInput.value) > (n - x + 1)) aInput.value = n - x + 1;
    if (parseInt(bInput.value) > (m - y + 1)) bInput.value = m - y + 1;
```

```
}

// 显示指定步骤
function showStep(step) {
    currentStep = step;

    // 更新步骤按钮样式
    stepBtns.forEach((btn, index) => {
        if (index + 1 === step) {
            btn.classList.add('step-active');
            btn.classList.remove('border-gray-300', 'text-gray-600');
        } else {
            btn.classList.remove('step-active');
            btn.classList.add('border-gray-300', 'text-gray-600');
        }
    });

    // 显示/隐藏查询控制
    if (step === 4) {
        queryControls.classList.remove('hidden');
        formulaDisplay.classList.remove('hidden');
    } else {
        queryControls.classList.add('hidden');
        formulaDisplay.classList.add('hidden');
        resultDisplay.classList.add('hidden');
    }

    // 更新解释文本
    updateExplanation();

    // 渲染矩阵
    renderMatrix();
}

// 更新解释文本
function updateExplanation() {
    switch (currentStep) {
        case 1:
            explanationText.textContent = "这是原始矩阵，矩阵的行和列都从1开始编号（第0行和
            break;
        case 2:
            explanationText.textContent = "行前缀和矩阵（rowsum）中，每个元素rowsum[i][j]
            break;
```

```
            case 3:
                explanationText.textContent = "二维前缀和矩阵（matrixsum）中，每个元素matrixsu
                break;
            case 4:
                explanationText.textContent = "通过二维前缀和矩阵，可以快速计算任意矩形区域的元
                break;
        }
    }

    // 渲染当前步骤的矩阵
    function renderMatrix() {
        let currentMatrix;
        let matrixName;

        // 确定当前要显示的矩阵
        switch (currentStep) {
            case 1:
                currentMatrix = matrix;
                matrixName = "原始矩阵 (matrix)";
                break;
            case 2:
                currentMatrix = rowsum;
                matrixName = "行前缀和矩阵 (rowsum)";
                break;
            case 3:
                currentMatrix = matrixsum;
                matrixName = "二维前缀和矩阵 (matrixsum)";
                break;
            case 4:
                currentMatrix = matrixsum;
                matrixName = "二维前缀和矩阵 (用于查询计算)";
                break;
        }

        // 构建矩阵HTML
        let html = `<div class="text-center mb-4"><h3 class="text-lg font-semibold">${matrix
        html += '<div class="matrix grid gap-0 border border-gray-300">';

        for (let i = 0; i <= n; i++) {
            html += '<div class="flex">';
            for (let j = 0; j <= m; j++) {
                let cellClass = 'matrix-cell';
```

```
            // 第0行和第0列用不同样式
            if (i === 0 || j === 0) {
                cellClass += ' bg-gray-100 text-gray-500';
            }

            // 在步骤4中高亮显示查询相关的单元格
            if (currentStep === 4) {
                const x = parseInt(xInput.value) || 1;
                const y = parseInt(yInput.value) || 1;
                const a = parseInt(aInput.value) || 1;
                const b = parseInt(bInput.value) || 1;

                const x2 = x + a - 1;
                const y2 = y + b - 1;

                // 高亮公式中用到的四个点
                if ((i === x2 && j === y2) ||
                    (i === x2 && j === y - 1) ||
                    (i === x - 1 && j === y2) ||
                    (i === x - 1 && j === y - 1)) {
                    cellClass += ' matrix-cell-selected';
                }
            }

            html += `<div class="${cellClass}">${currentMatrix[i][j]}</div>`;
        }
        html += '</div>';
    }

    html += '</div>';
    matrixContainer.innerHTML = html;
}

// 计算查询结果
function calculateQuery() {
    const x = parseInt(xInput.value) || 1;
    const y = parseInt(yInput.value) || 1;
    const a = parseInt(aInput.value) || 1;
    const b = parseInt(bInput.value) || 1;

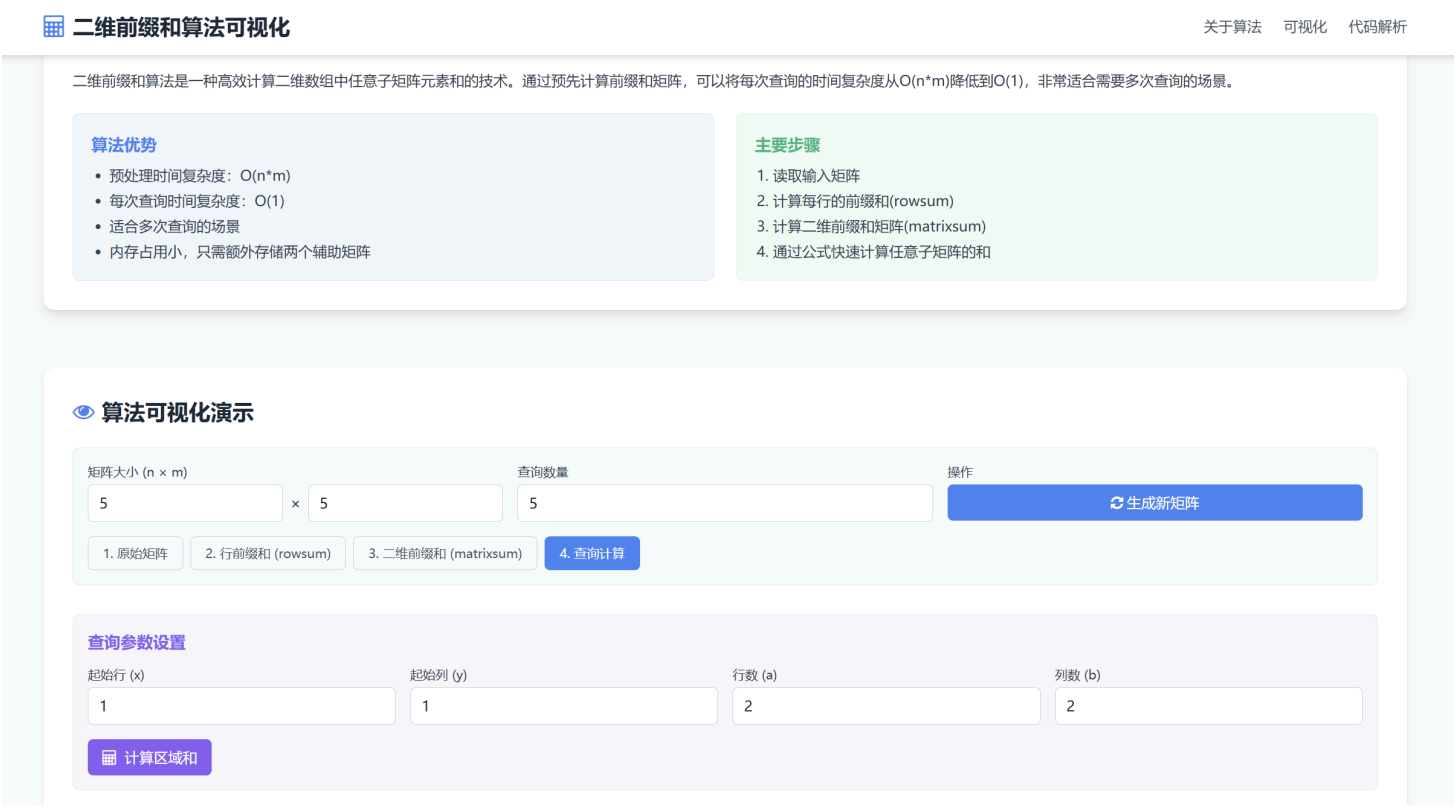
    // 计算区域右下角坐标
    const x2 = x + a - 1;
    const y2 = y + b - 1;
```

```
            // 检查是否超出范围
            if (x2 > n || y2 > m || x < 1 || y < 1 || a < 1 || b < 1) {
                alert('查询参数超出矩阵范围，请调整参数');
                return;
            }

            // 应用公式计算区域和
            const sum = matrixsum[x2][y2]
                      - matrixsum[x2][y - 1]
                      - matrixsum[x - 1][y2]
                      + matrixsum[x - 1][y - 1];

            // 更新结果显示
            resultValue.textContent = `区域和 = ${sum}`;
            resultDisplay.classList.remove('hidden');

            // 重新渲染矩阵以高亮显示相关单元格
            renderMatrix();
        }
    </script>
</body>
</html>
```



矩阵大小 (n × m)

5 × 5

查询数量

5

操作

生成新矩阵

1. 原始矩阵 | 2. 行前缀和 (rowsum) | 3. 二维前缀和 (matrixsum) | 4. 查询计算

查询参数设置

起始行 (x)

1

起始列 (y)

1

行数 (a)

2

列数 (b)

2

计算区域和

二维前缀和矩阵 (用于查询计算)

0	0	0	0	0	0
0	2	8	11	17	23
0	3	17	23	34	48
0	11	27	37	58	79
0	19	40	55	77	105
0	27	53	74	104	136

当前步骤解释

通过二维前缀和矩阵，可以快速计算任意矩形区域的元素和。输入查询参数（x,y,a,b），其中(x,y)是矩形的左上角坐标，a和b分别是矩形的高度和宽度。